

Implementation Errors of Model 1 (Background: Poisson, Efficiency: Binomial) in TRolke

Yu. Maslov^{*1} and D. Maximov^{†2}

¹Novosibirsk State University, Novosibirsk, Russia

²Budker Institute of Nuclear Physics, Novosibirsk, Russia

1 Introduction

In [1], a method for constructing confidence intervals using the profile likelihood approach is described. This method was implemented in the TRolke (ROOT class) for several models of signal-detection efficiency and background, which, despite being considered legacy and obsolete at present, continues to be used. Section 2.2 of [1] states that the system of equations for finding the suprema required to construct intervals in the model (denoted in TRolke as model 1)

$$X \sim \text{Pois}(\epsilon\mu + b), \quad Y \sim \text{Pois}(\tau b), \quad Z \sim \text{Bin}(m, \epsilon)$$

(where Poiss denotes the Poisson distribution and Bin denotes the binomial distribution) cannot be solved analytically and must be solved numerically; this statement is incorrect. During the implementation of the numerical solution of this system, a number of errors were also made.

This note presents an analytic solution of the aforementioned system and an implementation of this solution. A comparison is performed between the likelihood suprema obtained with the original implementation (containing an error) and those obtained with the corrected analytic approach. It is shown that the analytic implementation yields larger values over a set of parameters than the suprema produced by the original method.

In addition, the analytic solutions for the remaining models were verified; no errors in solving the equations or in implementing the supremum search were found.

2 Existence of an Analytic Solution

In the profile likelihood method for constructing confidence intervals, the following statistic is used:

$$\lambda(\pi_0|X) = \frac{\sup \{L(\pi_0, \theta|X); \theta\}}{\sup \{L(\pi, \theta|X); \pi, \theta\}},$$

where θ are nuisance parameters.

In model 1, the background is modeled by a Poisson distribution, and the efficiency by a binomial distribution:

$$L(\mu, b, \epsilon|x, y, z, m, \tau) = \frac{(\epsilon\mu + b)^x}{x!} e^{-(\epsilon\mu + b)} \frac{(\tau b)^y}{y!} e^{-(\tau b)} C_m^z \epsilon^z (1 - \epsilon)^{m-z},$$

$$\ln L(\mu, b, \epsilon|x, y, z, m, \tau) =$$

^{*}y.maslov@g.nsu.ru

[†]D.A.Maksimov@inp.nsk.su

$$= x \ln(\epsilon\mu + b) - (\epsilon\mu + b) + y \ln(\tau b) - \tau b + z \ln \epsilon + (m - z) \ln(1 - \epsilon) + C,$$

where C denotes the contribution from the factorials and the binomial coefficient. To find an extremum, one must solve the system of equations obtained by setting the partial derivatives with respect to the nuisance parameters equal to zero. In [1], the following system of nonlinear equations is given, together with the statement that no analytic solution can be obtained:

$$\frac{\partial}{\partial b} \log l(\mu, b, \epsilon | x, y, z) = \frac{x}{\epsilon\mu + b} - 1 + \frac{y}{b} - \tau \doteq 0$$

$$\frac{\partial}{\partial \epsilon} \log l(\mu, b, \epsilon | x, y, z) = \frac{x}{\epsilon\mu + b} - \mu + \frac{z}{\epsilon} - \frac{m - z}{1 - \epsilon} \doteq 0.$$

It should be noted that the system above contains a typo: the factor μ was dropped when differentiating with respect to ϵ . The correct system is:

$$\frac{\partial}{\partial b} \log l(\mu, b, \epsilon | x, y, z) = \frac{x}{\epsilon\mu + b} - 1 + \frac{y}{b} - \tau \doteq 0 \quad (1)$$

$$\frac{\partial}{\partial \epsilon} \log l(\mu, b, \epsilon | x, y, z) = \frac{x\mu}{\epsilon\mu + b} - \mu + \frac{z}{\epsilon} - \frac{m - z}{1 - \epsilon} \doteq 0. \quad (2)$$

Contrary to the statement in [1], this system can be solved analytically by reducing it to a quartic equation and applying standard methods for solving 4th-degree polynomials. From (1), it follows that

$$\epsilon\mu + b = \frac{x}{1 + \tau - \frac{y}{b}} \Rightarrow \epsilon = \frac{1}{\mu} \left(\frac{x}{1 + \tau - \frac{y}{b}} - b \right) = \frac{b}{\mu} \left(\frac{x - b(1 + \tau) + y}{b(1 + \tau) - y} \right).$$

Introduce the following notation: $R(b) = x + y - b(1 + \tau)$, $Q(b) = b(1 + \tau) - y$. Hence, $\epsilon = \frac{b}{\mu} \frac{R(b)}{Q(b)}$. Next, multiplying (1) by μ and subtracting it from (2), we obtain:

$$\left(\tau - \frac{y}{b} \right) \mu + \frac{z}{\epsilon} - \frac{m - z}{1 - \epsilon} = 0 \Rightarrow \left(\tau - \frac{y}{b} \right) \mu + z \frac{\mu Q(b)}{b R(b)} - \frac{m - z}{1 - \frac{b}{\mu} \frac{R(b)}{Q(b)}} = 0.$$

Introduce one more notation, $S(b) = \mu Q(b) - b R(b)$. Then:

$$1 - \frac{b}{\mu} \frac{R(b)}{Q(b)} = \frac{\mu Q(b) - b R(b)}{\mu Q(b)} = \frac{S(b)}{\mu Q(b)} \Rightarrow \left(\tau - \frac{y}{b} \right) \mu + z \frac{\mu Q(b)}{b R(b)} - (m - z) \frac{\mu Q(b)}{S(b)} = 0. \quad (3)$$

Multiply this equation by $b R(b) S(b)$. Then:

$$(\tau b - y) \mu R(b) S(b) + z \mu Q(b) S(b) - (m - z) \mu b Q(b) R(b) = 0.$$

It can be seen that this equation is a 4th-degree polynomial, which can be solved analytically in the general case. This equation can be brought to the form $Ab^4 + Bb^3 + Cb^2 + Db + E = 0$, where:

$$A = -\mu\tau(1 + \tau)^2,$$

$$B = \mu(1 + \tau)(m(1 + \tau) - \mu\tau(1 + \tau) + 2\tau x + y + 3\tau y),$$

$$C = \mu(-m(1 + \tau)(x + 2y) - (x + y)(\tau x + 2y + 3\tau y) + \mu(1 + \tau)(y + z + \tau(x + 3y + z))),$$

$$D = \mu y (m(x + y) + (x + y)^2 - \mu(x + 2\tau x + 2y + 3\tau y + 2(1 + \tau)z)),$$

$$E = \mu^2 y^2 (x + y + z).$$

3 Error in the Implementation of the Numerical Solution

Due to the statement that the system of equations (1), (2) is not analytically solvable, the following algorithm for finding the suprema was implemented in TRolke [2, 3]: instead of reducing the system to a single equation, the total derivative with respect to ϵ is set to zero (this derivative is computed by the LikeGradMod1 class method) using the bisection method (the bisection is implemented in the ProfLikeMod1 class method):

$$\frac{d \ln f}{d \epsilon} = \frac{x}{\epsilon \mu + b} (\mu + b') - (\mu + b') + \frac{y}{b} b' - \tau b' + \frac{z}{\epsilon} - \frac{m - z}{1 - \epsilon} = 0.$$

Further, from the original system of equations (1), (2) it follows that

$$-\frac{y\mu}{b} + \tau\mu = \frac{m - z}{1 - \epsilon} - \frac{z}{\epsilon}.$$

Let $\eta = \frac{z}{\epsilon} - \frac{m-z}{1-\epsilon}$, $\eta' = -\frac{z}{\epsilon^2} - \frac{m-z}{(1-\epsilon)^2}$. Then

$$-\frac{y}{b} + \tau = -\frac{\eta}{\mu} \Rightarrow b = \frac{y}{\tau + \eta/\mu} \Rightarrow b' = -\frac{b^2}{y\mu} \eta'.$$

However, in the technical implementation of this relation in the TRolke class, shown in Fig. 1, the following expressions for b and b' are used:

$$b = \frac{y}{\tau - \eta/\mu}, \quad b' = \frac{b^2}{y\mu} \eta'.$$

```

1060 //////////////////////////////////////////////////
1061 /// Gradient model likelihood
1062
1063 Double_t TRolke::LikeGradMod1(Double_t e, Double_t mu, Int_t x, Int_t y, Int_t z, Double_t tau, Int_t m)
1064 {
1065     Double_t eta, etaprim, bprime, f;
1066     eta = static_cast<double>(z) / e - static_cast<double>(m - z) / (1.0 - e);
1067     etaprim = (-1) * (static_cast<double>(m - z) / ((1.0 - e) * (1.0 - e)) + static_cast<double>(z) / (e * e));
1068     Double_t b = y / (tau - eta / mu);
1069     bprime = (b * b * etaprim) / mu / y;
1070     f = (mu + bprime) * (x / (e * mu + b) - 1) + (y / b - tau) * bprime + eta;
1071     return f;
1072 }
1073

```

Error

Figure 1: Technical implementation of the derivative evaluation.

The suprema computed by this method are smaller than those obtained using the analytic solution, as will be demonstrated explicitly below. After correcting this error, the bisection method finds local extrema with respect to ϵ that concur with the analytic solution; however, in most cases, these are not the desired supremum of the likelihood function.

4 Implementation of the Fix Based on the Analytic Solution

There exist different analytic methods for solving a quartic equation. The Ferrari method was chosen due to the relative simplicity of writing the solution in analytic form (see “Summary of Ferrari Method” in [4]). The implementation of the method is given in the listing below.

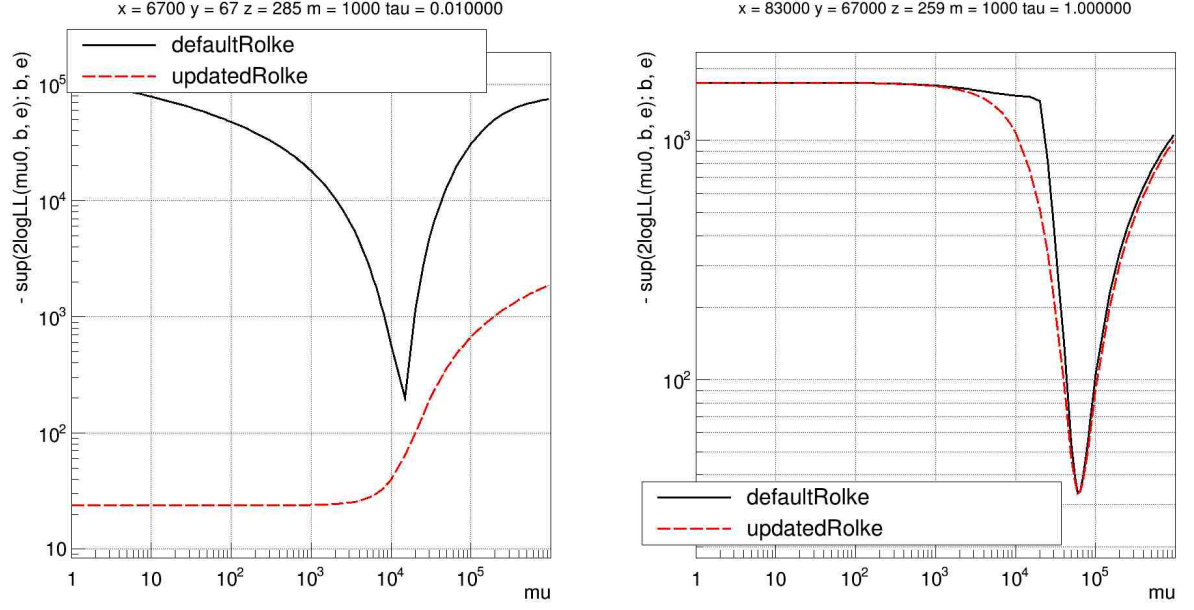


Figure 2: Comparison of the suprema obtained with the original and the analytic methods for different parameters.

Comparisons of the suprema for different parameters are shown in Figs. 2, 3, 4 (in all plots, the y -axis shows $-\sup 2 \ln L$ to enable the use of a logarithmic scale).

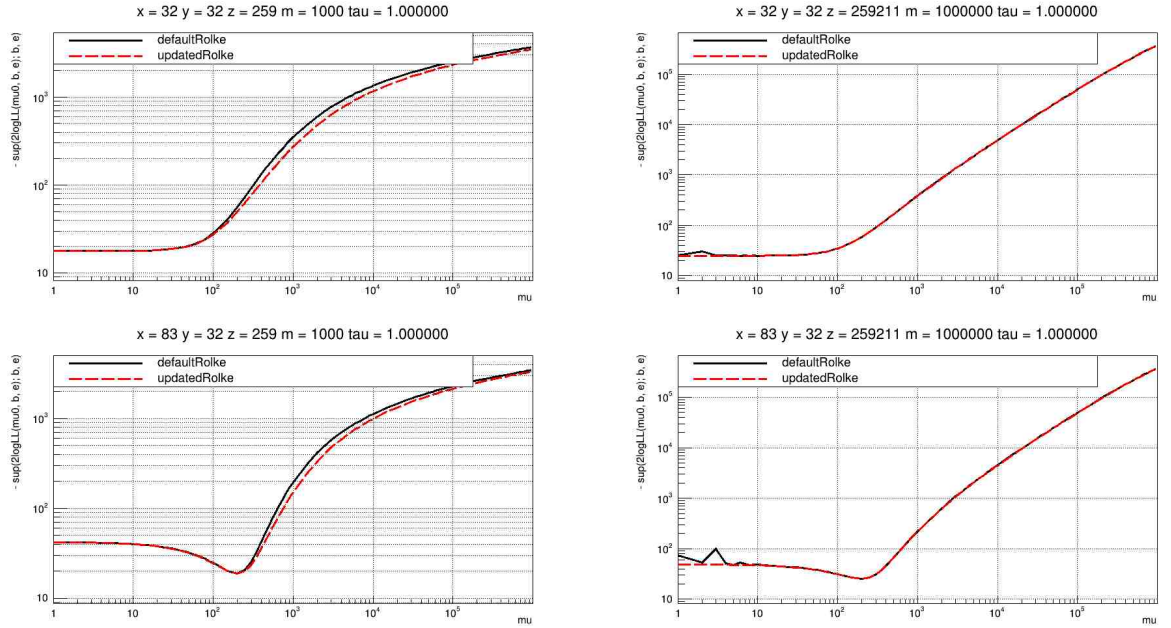


Figure 3: Comparison of the suprema obtained with the original and the analytic methods for different parameters.

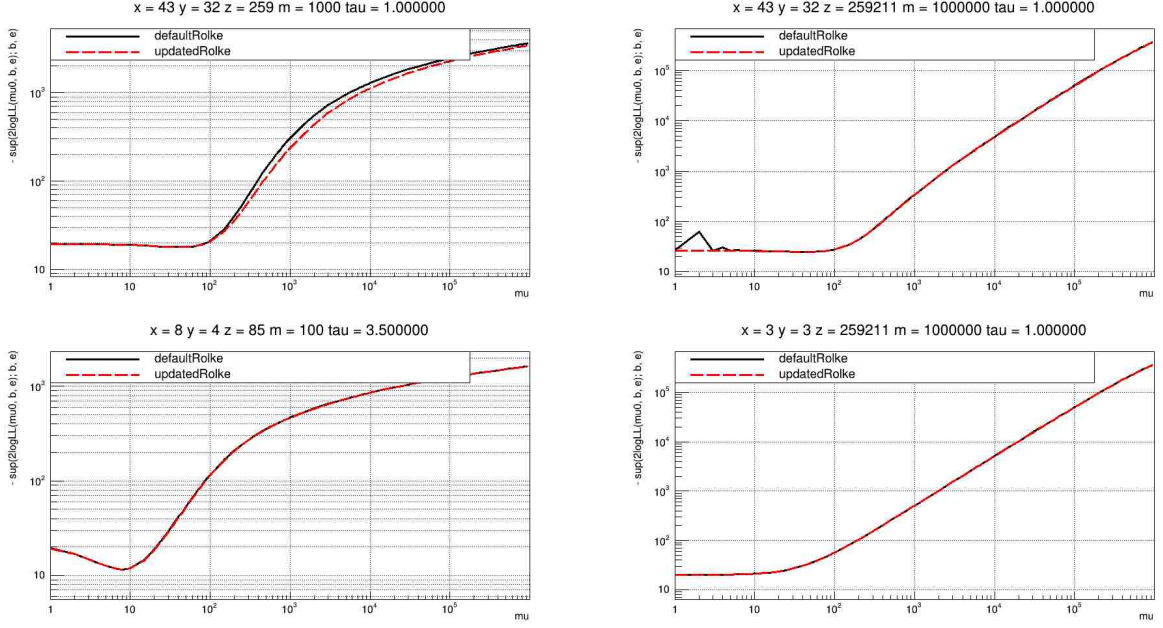


Figure 4: Comparison of the suprema obtained with the original and the analytic methods for different parameters.

It can be seen that the suprema obtained analytically take larger values than the suprema produced by the original method. It is also worth noting the absence of outliers in the analytic solution.

5 Uncertainties in the Current Analytic Solution

For some values of μ, x, y, z, m, τ , all solutions of the system may lie in an unphysical region. The current implementation does not account for this. The question of how this affects the coverage properties of the confidence intervals is not addressed.

References

- [1] Rolke W. A., López A. M., Conrad J. Limits and confidence intervals in the presence of nuisance parameters // Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment. – 2005. – Vol. 551. – No. 2–3. – pp. 493–503.
- [2] Lundberg J. et al. Limits, discovery and cut optimization for a Poisson process with uncertainty in background and signal efficiency: TRolke 2.0 // Computer Physics Communications. – 2010. – Vol. 181. – No. 3. – pp. 683–686.
- [3] ROOT (data analysis framework), <https://root.cern/>, accessed 2021-02-25
- [4] https://en.wikipedia.org/wiki/Quartic_equation

5.1 Code Listing of the Analytic Implementation for Finding the Supremum

```

void TRolke128_num::ProfLikeMod1Fix(--float128 mu, --float128 &b, --float128 &e,
    Int_t x, Int_t y, Int_t z, --float128 tau, Int_t m)
{
    --float128 A = - mu * tau * (1.0Q + tau)*(1.0Q + tau);
    --float128 B = mu * (1.0Q + tau) * (- mu * tau * (1.0Q + tau) + 2.0Q * tau * x + y +
    3.0Q * tau * y + m * (1.0Q + tau));
    --float128 C = mu * (- (x + y) * (tau * x + 2.0Q * y + 3.0Q * tau * y) -
    m * (1.0Q + tau) * (x + 2.0Q * y) + mu * (1.0Q + tau) * (y + z + tau * (x + 3.0Q * y + z)));
    --float128 D = mu * y * (((--float128) (x + y) * (x + y)) -
    mu * (x + 2.0Q * tau * x + 2.0Q * y + 3.0Q * tau * y + 2.0Q * (1.0Q + tau) * z) + m * (x + y));
    --float128 E = mu * mu * y * y * (x + y + z);

    auto poly4comp = [A, B, C, D, E] (--complex128 b) -> --complex128 {
        --complex128 b_2 = b * b;
        return A * b_2 * b_2 + B * b_2 * b + C * b_2 + D * b + E;
    };

    auto poly4rat = [A, B, C, D, E] (--float128 b) -> --float128 {
        --float128 b_2 = b * b;
        return A * b_2 * b_2 + B * b_2 * b + C * b_2 + D * b + E;
    };

    --float128 A_2 = A * A;
    --float128 B_2 = B * B;

    --float128 alpha = -3.0Q * B_2 / (8.0Q * A_2) + C / A;
    --float128 alpha_2 = alpha * alpha;
    --float128 beta = B_2 * B / (8.0Q * A_2 * A) - B * C / (2.0Q * A_2) + D / A;

    --float128 gamma = - 3.0Q * B_2 * B_2 / (256.0Q * A_2 * A_2) +
    B_2 * C / (16.0Q * A_2 * A) - B * D / (4.0Q * A_2) + E / A;

    --float128 P = - alpha_2 / 12.0Q - gamma;
    --float128 Q = - alpha_2 * alpha / 108.0Q + alpha * gamma / 3.0Q - beta * beta / 8.0Q;

    --complex128 R = - Q / 2.0Q + csqrtq(Q * Q / 4.0Q + P * P * P / 27.0Q); // taken + sqrt

    --complex128 U = cpowq(R, 1.0Q / 3.0Q);

    --complex128 y_eq = - 5.0Q / 6.0Q * alpha + U - P / (3.0Q * U); // assuming that U != 0;
    --complex128 W = csqrtq(alpha + 2.0Q * y_eq);

    --complex128 back[4] = {-1.0Q, -1.0Q, -1.0Q, -1.0Q};
    --complex128 eff[4] = {10.0Q, 10.0Q, 10.0Q, 10.0Q};
    --float128 minusPlusS[4] = { 1.0Q, 1.0Q, -1.0Q, -1.0Q};
    --float128 minusPlusT[4] = { 1.0Q, -1.0Q, 1.0Q, -1.0Q};

    --float128 maxLL = (-1.0Q) * FLT128_MAX;
    --float128 backResult = 0;
    --float128 effResult = 0;

    // loop through signs
    for (int j = 0; j < 4; j++) {
        back[j] = - B / (4.0Q * A) + (minusPlusS[j] * W + minusPlusT[j] *
        csqrtq(-(3.0Q * alpha + 2.0Q * y_eq + minusPlusS[j] * 2.0Q * beta / W))) / 2.0Q;
        eff[j] = CalcEff(mu, back[j], x, y, z, tau, m);

        --complex128 poly4compVal = poly4comp(back[j]);

        if (crealq(eff[j]) <= 1.0Q && crealq(eff[j]) >= 0.0Q && crealq(back[j]) >= 0.0Q) {
            --float128 currentLL = LikeMod1(mu, crealq(back[j]), crealq(eff[j]), x, y, z, tau, m);
            if (currentLL >= maxLL) {
                backResult = crealq(back[j]);
                effResult = crealq(eff[j]);
                maxLL = currentLL;
            } else {
                std::cout << "CurrentLL-<-maxLL" << std::endl;
            }
        } else {
            std::cout << "e-or/and-b-are-out-of-range" << std::endl;
        }
    }

    std::cout << "backResult:-" << (double) backResult << "-effResult:-" << (double) effResult << std::endl;
    b = backResult;
    e = effResult;
}

```